

Served-Realization Tree Verification for Recurrent Hybrid Language Models

Anonymous Authors
Anonymous Institution

Abstract—Speculative decoding with token trees is well understood for attention-only targets, but recurrent hybrid language models add a second correctness boundary: each candidate branch must carry the recurrent state that native sequential decoding would have produced on that branch. We present GDN Tree-Scan, a served target verifier for Qwen 3.6-27B-FP8 inside vLLM. The verifier combines a forked FlashAttention-2 decode path with tree ancestry bias, a branch-local Triton Gated-DeltaNet scan/replay kernel, device-side multidraft commitment, and explicit durable-state publication rules. The evidence boundary is narrow: the current lossless gate is a sampled clear-margin p-rescore that closes within the observed native recurrent-oracle flip floor, not a full distribution-distance or request-cluster-bootstrap proof. In that same clean B=1, temperature-0.6, four-task SWE/Codex gate, the six-node root-branch tree reaches 23.88 token-weighted decode tokens/s versus 18.80 for native E5, a 27.0% decode-throughput gain. The same run is only 4.0% faster under a per-request-equal latency view, and end-to-end task wall time remains prefill-heavy. We report the verifier contract, implementation seams, evidence gates, and failed routes that define this scoped decode-throughput result.

Index Terms—speculative decoding, recurrent models, Gated DeltaNet, vLLM, FlashAttention, ML systems

I. INTRODUCTION

Speculative decoding accelerates autoregressive generation by drafting candidate tokens and asking the target model to verify them in parallel. Tree-shaped speculative decoding generalizes a single draft chain into a candidate token tree, improving the opportunity set for each target forward pass while preserving the target distribution under the verifier acceptance rule [1]–[4]. For attention-only Transformer stacks, the central verifier problem is largely an ancestry problem: row i may attend to the prompt and its ancestors, but not to sibling branches [1], [3].

Recurrent hybrid models break that simplification. Qwen 3.6-style hybrid stacks contain many Gated-DeltaNet (GDN) recurrent layers in addition to softmax-attention layers, so a valid tree row needs two path-local facts: attention ancestry and recurrent state lineage [5], [6]. The verifier must ensure that a node’s GDN state equals the state native sequential decode would have produced along that node’s root-to-node path. If sibling rows share a packed recurrent accumulator, a correct attention mask can still verify tokens from an impossible recurrent history [7], [8].

This paper reports a served verifier for a recurrent hybrid target inside vLLM. The contribution is not “tree speculative decoding” in the abstract. It is served-realization equivalence as an implementation target: forked FlashAttention-2 (FA2)

tree bias for attention, branch-local GDN scan and accepted-chain replay for recurrence, and vLLM state publication that makes rejected branch state non-durable [7], [9]–[11].

The measured result is deliberately narrow. In a clean B=1, temperature-0.6 deployment trace over four SWE/Codex tasks, cat6root, a six-node tree with a depth-5 native spine plus one root sibling, reaches 23.8768 token-weighted decode tokens/s versus 18.7962 for native E5, a 27.0% decode-throughput gain [12], [13]. The per-request-equal latency view is 18.5056 versus 17.8018 tokens/s, or +4.0%; this is useful but underweights long decode turns where most generated tokens occur [14]. Both cat6root and cat9 close within the observed native recurrent-oracle flip floor in a 40-turn B=1 p-rescore, but full distribution-distance evidence, request-level bootstrap, more seeds, B=4, and Stage D timing remain open gates [15], [16].

The paper makes three concrete contributions:

- A served verifier contract for recurrent hybrid token trees: attention ancestry, path-local GDN state, SpecInfer-style residual commitment, and accepted-chain-only durable publication.
- A vLLM/FA2/GDN implementation path that preserves native serving machinery where possible and adds tree-specific state only at explicit seams.
- An evidence ledger that separates clean B=1 decode throughput, per-request latency, recurrent-oracle p-rescore, end-to-end wall caveats, and contaminated or still-open measurements.

II. BACKGROUND AND RELATED WORK

Speculative decoding uses a cheaper draft path to propose future tokens and verifies them with the target model. SpecInfer introduced tree-based speculative inference and parallel verification for LLM serving, while Sequoia, EAGLE-2, and OPT-Tree study hardware-aware, dynamic, or adaptive draft-tree structure [1]–[4]. These systems establish the candidate-tree and verifier-acceptance framing, but the usual target-forward assumption is too high-level for a served recurrent hybrid.

SSTree is the closest prior work for stateful tree decoding. It extends tree speculative decoding to SSM and hybrid architectures by exploiting accumulated state-transition structure [17]. That framing transfers directly: stateful models need tree-aware state construction, not only tree-aware attention. The missing layer for this artifact is realization. The served target path has fp8/bf16 seams, branch co-residency, a specific FA2

attention implementation, GDN state writeback, and rejection sampling inside vLLM [8], [18].

Recent hybrid serving and SSM speculation work sharpens the same boundary. SpecMamba identifies hidden-state backtracking, tree-verification incompatibility, and draft/target workload mismatch for Mamba speculative decoding, then targets an FPGA dataflow [19]. Component-aware self-speculation shows that hybrid component structure matters: Falcon-H1 and Qwen3.5 behave differently because of how SSM/linear-attention components are integrated [20]. These papers explain why hybrid models create new verifier questions, but they do not close the vLLM/FA2/GDN served-realization path.

SGLang is the closest adjacent production comparison and should be treated carefully. Its hybrid-model writeup separates KV cache from request-level Mamba state, introduces MambaRadixCache, and handles speculative verification with independent state slots and parent tracing [21]. Its Qwen3-Next documentation and speculative-decoding documentation show active support for GDN/MTP-style hybrid models [22], [23]. That is not an apple-to-apple fused-kernel baseline for this paper: much of the mechanism is cache/scheduler-layer work, while this artifact’s shipped contribution is lower in the stack, at the vLLM, forked-FA2, branch-local GDN scan/replay, and publication boundary [6], [24].

III. VERIFIER CONTRACT

GDN Tree-Scan verifies an arbitrary candidate tree, not only the native MTP drafter used in the current implementation. The input contract is a prefix state, a parent array, candidate token ids, and sampling state. The output contract is stronger: after verification and commitment, the next served decode event must start from the same observable token prefix and recurrent state that native sequential decode would have reached on the accepted tokens, up to the observed native recurrent-oracle numerical floor [6], [15].

The contract can be summarized as an engineering invariant. Assume the tree contains only candidate ids and parent links; attention rows see only prefix plus ancestors; every GDN row is computed from its parent row’s recurrent state; the committer applies the same residual rule as the target verifier; and durable state is replayed or published only for the accepted chain. Then the next served decode event has the same intended token-prefix and recurrent-state boundary as native sequential decode on the accepted prefix. The current evidence checks this boundary through recurrent-oracle p-rescore and targeted scan/replay gates rather than through a full distribution-distance proof [1], [7], [15].

IV. SYSTEM DESIGN

Figure 1 shows the served path. The drafter currently uses Qwen’s native MTP head: root and spine tokens come from the same logits tensor, with runner-up leaves read as top- k candidates. The verifier should not depend on that drafter. Its stable boundary is a tree descriptor with parent indices,

Algorithm 1 Tree verification on a recurrent hybrid target

Require: prefix state S_0 , candidate tree $T = (V, E)$, token ids x_i , sampling state

- 1: Pack candidate ids, parent indices, and ancestry masks into vLLM decode metadata.
- 2: Run softmax-attention layers with FA2 tree bias so row i sees prefix and ancestors only.
- 3: **for** each GDN layer **do**
- 4: **for** each node i in topological order **do**
- 5: Load $S_{\text{parent}(i)}$ from S_0 or branch-local `h_cache`.
- 6: Compute $S_i, o_i \leftarrow \text{GDNStep}(S_{\text{parent}(i)}, x_i)$.
- 7: Store S_i in `h_cache[i]` for children.
- 8: **end for**
- 9: **end for**
- 10: Apply the residual speculative acceptance rule on device.
- 11: Replay accepted path in native order and publish only accepted GDN/conv state.
- 12: **return** committed tokens and durable recurrent state for the next decode event.

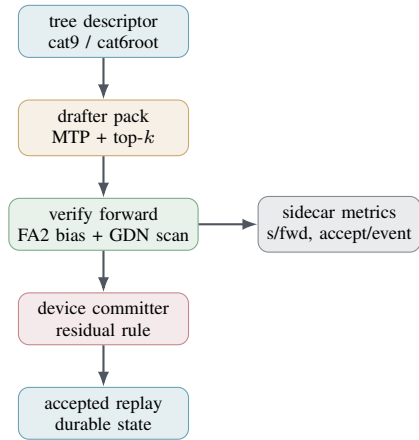


Fig. 1. The tree shape is a cross-layer contract. Candidate packing, attention metadata, recurrent scan, commitment, replay, and metrics must consume the same descriptor.

candidate ids, visibility masks, and accepted-path rows that can be replayed into native state [25], [28].

The attention side needed a real FA2 fork. vLLM’s generic tree attention path could express an additive ancestry bias through a Triton `unified_attention` route, but that route was not the same realization as native FA2 decode. The forked path adds an fp32 `tree_bias` after the QK score tile and before softmax inside the FA2 varlen decode path, while keeping prefill on native FA2 [9], [11], [26]. This makes the attention half of tree verification a minimal extension of the native served carrier.

The recurrent side needed branch-local state and publication discipline. During verification, each candidate row computes a GDN post-state from its parent row, not from the previous packed row. During commitment, the accepted chain is replayed in native order before durable state is written back. Rejected rows can influence acceptance decisions, but they

TABLE I

COMPARISON LAYER FOR PRIOR AND ADJACENT WORK. THE GOAL IS TO AVOID PROMOTING ADJACENT SERVING WORK INTO A DIRECT FUSED-KERNEL BASELINE.

Line of work	What transfers	What breaks for this target	What GDN Tree-Scan adds
SpecInfer, Sequoia, EAGLE-2, OPT-Tree	Candidate trees, verifier acceptance, dynamic or hardware-aware shape selection [1]–[4].	The target verification forward is assumed to realize the target distribution once attention ancestry is correct.	Recurrent-state verification and deployment lossless gates.
STree	Stateful models need tree-aware state construction [17].	Algebraic state sharing is not the same as matching native served GDN under bf16/tp8 and vLLM writeback.	A GDN scan/replay kernel gated against native/deployment oracles.
SpecMamba	Hidden-state backtracking and tree verification are first-class SSM obstacles [19].	FPGA dataflow and FIFO verification are not a served GPU/vLLM path for Qwen GDN.	Fused GPU kernels, FA2/vLLM compatibility, and deployment gates.
Component-aware self-speculation	Hybrid component structure determines speculative viability [20].	Research Python implementation leaves fused CUDA, batched verification, and cache sharing as systems gaps.	A fused served path for Qwen GDN tree verification.
SGLang hybrid serving	Adjacent production mechanisms: dual memory pools, MambaRadixCache, parent tracing, MTP/EAGLE surfaces [21]–[23].	Cache/scheduler-layer mechanisms are not directly comparable to this fused vLLM GDN tree-scan kernel.	A different layer of realization: vLLM + forked FA2 tree bias + branch-local GDN scan/replay + observed recurrent-oracle gates.

TABLE II
ENGINEERING FORM OF THE ACCEPTED-STATE VERIFIER CONTRACT.

Part	Formal content	Served implementation
Inputs	Prefix state, parent array, candidate ids, sampling state.	vLLM tree packing plus multidraft metadata [25].
Attention	Row i sees prompt tokens and ancestors of i , not siblings.	Forked FA2 adds <code>tree_bias</code> before softmax; prefill stays native [11], [26].
GDN	$S_i = \text{GDNStep}(S_{\text{parent}(i)}, x_i)$.	Triton GDN tree scan with branch-local <code>h_cache</code> [7].
Acceptance	Apply the same residual speculative rule as target verification.	Device multidraft commit at temperature 0.6 [27].
Publication	Only accepted-prefix tokens and their state become durable.	Accepted-chain replay and vLLM state writeback/remap [7], [25].

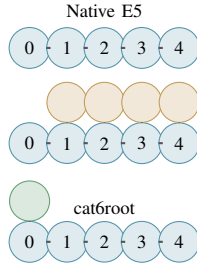


Fig. 2. The evaluated shapes. cat9 adds runner-up leaves to the depth-5 spine; cat6root keeps the spine and adds only a root sibling, reducing verifier work while preserving most of the acceptance gain.

must not become future model memory [7], [24], [25].

V. GDN SCAN AND REPLAY KERNEL

The GDN recurrence is a stateful linear-attention update. For one value tile, the served kernel implements the same

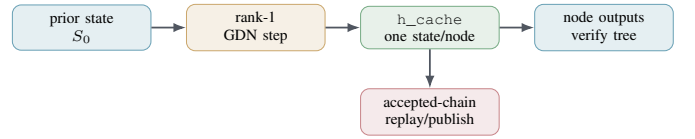


Fig. 3. The scan verifies all candidate nodes with parent-indexed recurrent state. Replay re-executes only the accepted chain and publishes durable GDN/conv state.

rank-1 update body used by the native path:

$$S'_t = \exp(g_t) S_{t-1}, \quad (1)$$

$$\Delta_t = \beta_t (v_t - S'_t k_t), \quad (2)$$

$$S_t = S'_t + \Delta_t k_t^\top, \quad (3)$$

$$o_t = S_t q_t. \quad (4)$$

For a tree node i , the invariant is $S_i = \text{native_scan}(\text{tokens}(\text{root} \rightarrow i))$. This is simple to state and easy to violate in a packed GPU implementation [5], [7], [8].

The kernel keeps a post-state for each padded node in `h_cache`. That table has shape $[N_{\text{PAD}}, BV, DIM_K]$ in fp32, where BV is the value-channel tile width. This is the main local resource constraint: native-match geometry such as $BV32$, one warp, and three stages can spill for a full $N_{\text{PAD}} = 16$ tree because the per-node state table becomes too large [7], [29]. The deployed scan uses $BV16$, eight warps, and pad16 for cat9; cat6root uses the smaller six-node tree and pad8-style shape economics while sharing the same verifier contract [28].

The numeric seams are also explicit. The scan-alignment notes track beta’s bf16 round trip, q/k normalization by division rather than reciprocal-square-root, and the carried-state bf16 boundary after output computation [7], [29]. These details are not cosmetic. In speculative decoding, a small branch-local numerical difference matters if it changes a clear-margin argmax or a residual sampling decision after many recurrent and attention layers [15], [30].

VI. EXPERIMENTAL SETUP

The reported experiment uses Qwen 3.6-27B-FP8, a hybrid model with 48 GDN recurrent layers and 16 softmax-attention layers, served from `/models/qwen3.6-27b-fp8` as `qwen3.6-27b` on GB10 [6], [31]. The source-of-truth runtime is a `vllm/vllm-openai` image pinned to digest `sha256:3dbe092ec5b2cef63b6104d33fa75d6ce53a7870962529ada69f78bbbc38e776`, corresponding to `vLLM 0.19.2rc1.dev134+gfe9c3d6c5`, plus the forked FA2 `.so` swap and the boot-time tree-GDN patcher [32], [33]. The clean B=1 tree arms use the forked FA2 tree-server launcher; `cat9` also has a locked wrapper, native E5 uses the native speed launcher, and the B=1 deploy wrapper pins `GPU_UTIL=0.82`, `MAX_NUM_SEQS=1`, and `MAX_MODEL_LEN=131072` [28], [34]. The receipts pin the mounted model path but not an immutable upstream model snapshot hash; that remains a packaging requirement before submission. The workload is four SWE/Codex tasks in clean B=1 mode, temperature 0.6, with true sequential windows and `num_running` near zero [12], [14].

Speed and lossless evidence are measured in separate modes. Speed runs disable heavy capture/oracle probes; lossless runs enable recurrent-oracle capture and p-rescore [15], [35]. This separation is not bookkeeping. Several earlier readings were confounded by raw prompt probes, B=4 co-residency, instrumentation overhead, or denominator mistakes, so the final result reports token-weighted decode throughput, per-request latency, lossless p-rescore, and end-to-end wall caveats as different measurements [12], [36].

Table III is the core speed result. `cat6root` commits 4.821 tokens/event versus 4.112 for native E5, a 17.2% committed-token gain, while keeping verify-forward time essentially native. The older +4.0% number is still true for the per-request-equal latency view, but that view weights a short tool-call turn like a long edit turn. The token-weighted aggregate is the fair decode-throughput basis for this scoped mechanism result [12], [14].

The request anatomy explains why these metrics differ. In the `cat6root` B=1 trace, 24.7% of turns have at most 64 output tokens, but those turns account for only 3% of output tokens; they are mostly short shell-tool calls in the Codex agent loop [14]. Speculative decoding helps more on longer turns, so per-request-equal averaging undercounts the decode-token gain. This does not imply a 27% end-to-end task-wall gain, because the same workload repeatedly sends roughly 11k–14k prompt tokens and prefix caching is off for this hybrid setup [14].

VII. LOSSLESS EVIDENCE AND MEASUREMENT DISCIPLINE

The empirical lossless claim in this artifact is closure within the observed native recurrent-oracle flip floor. The exact B=1 depth-5 p-rescore compares each arm against its own no-spec recurrent oracle under temperature 0.6. Native E5 has 630 clear-margin flips over 4,883 positions, or 12.90%; `cat9` has 645 over 4,858, or 13.28%; `cat6root` has about 636 over about

4,860, or 13.09% [15]. Both tree arms are within the native E5 floor by less than one position-level standard error, but this is not a TV-distance estimate and not a request-cluster bootstrap.

The statistical caveat matters. The p-rescore sample has roughly 4.9k positions per arm, but positions inside one SWE/Codex turn are correlated. The result is therefore clean B=1 closure for the observed sample, not a universal guarantee across all seeds, batch sizes, or prompt distributions [6], [15]. Request-cluster bootstrap, additional seeds, and full distribution-distance measurements are follow-up gates.

The measurement infrastructure is part of the contribution because it prevented false wins and false losses. The canonical formulas are:

$$s/fwd_gpu = \frac{\text{decode forward GPU seconds}}{\text{pure decode drafts}}, \quad (5)$$

$$\text{accept/event} = \frac{\text{accepted draft tokens}}{\text{draft events}}, \quad (6)$$

$$\text{deploy TPS} = \frac{\text{request decode tokens}}{\text{request decode seconds}}. \quad (7)$$

Probe separation, deployment-path-only measurement, token weighting, and oracle-mode separation are all needed to make the numbers interpretable [12], [14], [35].

VIII. FAILURE MODES AND VALIDITY LIMITS

The HTML page contains a long failure library. For an MLSys paper, the important function of that library is to explain why the final verifier has its particular contract. Attention-mask-only trees failed because GDN recurrence still carried sibling-contaminated state. A shared packed recurrent accumulator was invalid for co-resident branches. STree-style state sharing transferred as a problem lens, not a served implementation proof. Triton `TREE_ATTN` remained useful as a diagnostic/fallback, but the reported decode path had to stay on forked FA2 [8], [18], [26].

Other routes were not disproven as mathematics, only rejected as the proof path for this artifact. WY-style whole-tree algebra remains a possible future speed route, but it failed the served-proof gate for this delivery. Literal 0.0 max-abs was too strict because the native recurrent oracle itself has a numerical floor. B=4 speed reads were quarantined because a strict tree superset should not accept fewer spine tokens than native; the observed contaminated read violated that invariant [15], [36], [37].

The remaining open gates are clear. First, B=4 needs a refreshed carrier-clean deployment run because scheduler, prefill, and co-residency change the surface. Second, Stage D overhead attribution is not settled: drafter, verify forward, and isolated committer compute are near E5, but live step-wall attribution needs direct phase timers [16], [38]. Third, end-to-end task wall time is prefill-heavy with prefix caching off, so the +27% decode gain is not a full task-wall claim [14].

IX. CANONICAL CONFIGURATION

The shipped `cat9` and `cat6root` arms share one tree-serving configuration. The only intended per-arm difference is the

TABLE III
CLEAN B=1 DECODE RESULT. THROUGHPUT DELTA IS TOKEN-WEIGHTED DECODE TPS RELATIVE TO NATIVE E5.

Arm	Tree	Per-request TPS	Token-weighted TPS	Delta	s/fwd_gpu	Accept/event
Native E5	Native 5-step MTP spine	17.8018	18.7962	baseline	0.136955	3.1122
cat9	9-node caterpillar: 5-spine + 4 leaves	18.4366	22.6307	+20.4%	0.144030	3.6406
cat6root	6-node root branch: 5-spine + root sibling	18.5056	23.8768	+27.0%	0.137719	3.8214

TABLE IV
LOSSLESS CLOSURE GATES AND INTERPRETATION.

Gate	Checked	Status
Depth-5 p-rescore	E5/cat9/cat6root temp-0.6 recurrent oracle.	Within native floor [15].
GDN scan A/B	$BV16 = BV32 = 0.0$ in controlled scan.	Scan alone is clean [29].
Native path oracle	Path-local state versus native decode order.	Core invariant [7].
Device committer	Same residual-mix rule on device.	Residual-rule gate [27].
Broad cat9 floor	Big-denom deployment consolidation.	Overlapping floor intervals [15].

TABLE V
LOAD-BEARING CONFIGURATION CHOICES IN THE SHIPPED TREE PATH.

Surface	Choice
Tree mode	FR10_ENABLE_TREE_GDN=1, FR10_DECODE_MODE_DEFAULT=tree_mtp.
Attention	FR13_FA2_TREE_BIAS=1, FR13_FA2_PREFILL_NATIVE=1.
GDN batch variance fix	LUMO_FB_KERNEL_ROWS=1, LUMO_FB_PROJ_PAD_ROWS=16.
Committer	FR13_DEVICE_MULTIDRAFT=1 default-on for tree arms.
Safety	FR10_ALLOW_LINEAR_FALLBACK unset.
Operational B=1	GPU_UTIL=0.82, MAX_NUM_SEQS=1.

speculative tree shape: cat9 is a 9-node depth-5 spine plus leaves, while cat6root is a 6-node depth-5 spine plus root sibling [28]. Table V lists the load-bearing choices most likely to invalidate the result if changed.

The committer row follows the later bake addendum: the older canonical-config catalog still lists FR13_DEVICE_MULTIDRAFT as diagnostic/off, while the addendum records that it was baked default-on for tree arms after verifying that native E5 does not read the tree committer gate [28], [39]. Some flags remain explicit on purpose. The FA2 tree-bias and native-prefill flags are default-off in the patcher to preserve patch idempotency anchors; baking them would require a separate re-patch verification task [28], [39]. The fixed-row in-projection pad should not be baked globally because it can perturb native E5 bytes, which define the recurrent-oracle baseline [39].

X. CONCLUSION

GDN Tree-Scan turns a token tree into a served verifier for a recurrent hybrid language model by treating recurrent state as part of the tree contract. Attention ancestry is necessary, but not sufficient. The shipped path adds FA2 tree bias for softmax layers, branch-local GDN scan/replay for recurrent layers, and accepted-chain-only state publication for vLLM. Under the

clean B=1 deployment gate, cat6root improves token-weighted decode throughput by 27.0% while closing within the observed native recurrent-oracle flip floor [12], [15]. The result should be read with its caveats attached: decode-only, B=1, 40-turn p-rescore, no request-cluster bootstrap, no full distribution-distance estimate, prefill-heavy workload, and open B=4/Stage D follow-up.

ARTIFACT AND SOURCE TRAIL

The source blueprint is the live GitHub Pages paper, fetched and byte-matched to the local gh-pages worktree on June 18, 2026 [6]. This provenance is intentionally separated from the main technical framing. The main source receipts are the deployed TPS correction, B=1 lossless verdict, request anatomy, Stage D localization, vLLM source-of-truth ledger, launchers, FA2 fork correction, GDN tree kernel, vLLM tree wiring patch, device multidraft committer, measurement harness, canonical config ledger, and bake addendum [7], [12], [14]–[16], [25]–[28], [32]–[35], [39].

REFERENCES

- [1] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, “Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification,” 2024.
- [2] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, “Sequoia: Scalable, robust, and hardware-aware speculative decoding,” 2025.
- [3] Y. Li, F. Wei, C. Zhang, and H. Zhang, “Eagle-2: Faster inference of language models with dynamic draft trees,” 2024.
- [4] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang, “Opt-tree: Speculative decoding with adaptive draft tree structure,” 2025.
- [5] S. Yang, J. Kautz, and A. Hatamizadeh, “Gated delta networks: Improving mamba2 with delta rule,” 2025.
- [6] Lumo FlyWheel Contributors, “Lossless-within-floor GDN Tree-Scan: Served-Realization Tree Verification,” https://macoredroid.github.io/Lumo_FlyWheel/gdn-tree-scan.html, 2026. GitHub Pages blueprint, last updated 2026-06-17; fetched 2026-06-18.
- [7] Lumo FlyWheel Contributors, “GDN tree-scan kernel source,” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/src/lumo_flywheel_serving/fr10_gdn_tree_kernel.py, 2026.
- [8] Lumo FlyWheel Contributors, “FR10 phase-4 real-tensor de-risk,” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/docs/reports/auto_research/fr10-phase4-real-tensor-derisk-20260603.md, 2026.
- [9] T. Dao, “Flashattention-2: Faster attention with better parallelism and work partitioning,” 2023.
- [10] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” 2023.
- [11] Lumo FlyWheel Contributors, “FA2 tree-bias patch source,” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/scripts/fr13_patch_fa2_tree_bias.py, 2026.

- [12] Lumo FlyWheel Contributors, “FR13: cat6 deploy decode-throughput is +27%, not +4%.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/research/fr13_workflows/deploy_tps_resolved_plus27.md, 2026.
- [13] Lumo FlyWheel Contributors, “B=1 depth-5 speed verdict raw JSON.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/research/fr13_workflows/b1_depth5_speed_verdict.raw.json, 2026.
- [14] Lumo FlyWheel Contributors, “FR13: anatomy of the Codex plus SWE-Verified request stream.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/research/fr13_workflows/codex_swe_request_anatomy.md, 2026.
- [15] Lumo FlyWheel Contributors, “FR13 B=1 depth-5 lossless verdict.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/research/fr13_workflows/b1_depth5_lossless_verdict.md, 2026.
- [16] Lumo FlyWheel Contributors, “FR13 Stage D: localize cat6-vs-E5 step overhead.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/research/fr13_workflows/stage_d_localization_vs_e5.md, 2026.
- [17] Y. Wu, Z. Qin, A. Wong, and S. Soatto, “Stree: Speculative tree decoding for hybrid state-space models,” 2025.
- [18] Lumo FlyWheel Contributors, “FR13 tree-verify literature mapping.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/FR13_TREE_VERIFY_LITERATURE.md, 2026.
- [19] L. Zhong, S. Xu, H. Wen, T. Xie, Q. Guo, Y. Wang, and M. Li, “Specmamba: Accelerating mamba inference on fpga with speculative decoding,” 2025.
- [20] H. Borobia, E. Seguí-Mas, and G. Tormo-Carbó, “Component-aware self-speculative decoding in hybrid language models,” 2026.
- [21] SGLang Team, “Hybrid Models Meet SGLang: More than Full Attention.” <https://pytorch.org/blog/hybrid-models-meet-sglang-more-than-full-attention/>, 2025.
- [22] SGLang Project, “Qwen3-Next SGLang documentation.” <https://lmsysorg.mintlify.app/cookbook/autoregressive/Qwen/Qwen3-Next>, 2026.
- [23] SGLang Project, “Speculative Decoding Documentation.” https://sgl-project.github.io/advanced_features/speculative_decoding.html, 2026.
- [24] SGLang Project, “Hybrid-GDN MTP speculative decoding is not lossless on Ascend NPU.” <https://github.com/sgl-project/sglang/issues/25587>, 2026.
- [25] Lumo FlyWheel Contributors, “vLLM GDN tree wiring patch.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/scripts/fr10_phase4_patch_vllm_tree_gdn.py, 2026.
- [26] Lumo FlyWheel Contributors, “FR13 FA2 fork is decode-kernel correction.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/FR13_FA2_FORK_IS_DECODE_KERNEL_CORRECTION.md, 2026.
- [27] Lumo FlyWheel Contributors, “Device multidraft committer source.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/scripts/fr13_device_multidraft_kernel.py, 2026.
- [28] Lumo FlyWheel Contributors, “FR13 canonical config: cat6 and cat9 shipped deploy.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/research/fr13_workflows/cat6_cat9_canonical_config.md, 2026.
- [29] Lumo FlyWheel Contributors, “FR13 scan/SRAM/N-PAD verdict.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/FR13_SCAN_BITEXACT_SRAM_NPAD_VERDICT_BIND.md, 2026.
- [30] Lumo FlyWheel Contributors, “FR13 node-5 ladder diffuse verdict.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/FR13_NODE5_LADDER_DIFFUSE_BIND.md, 2026.
- [31] Qwen Team, “Qwen3-Next-80B-A3B-Instruct.” <https://qwen.ai/blog?from=research.latest-advancements-list&id=4074cca80393150c248e508aa62983f9cb7d27cd>, 2025.
- [32] Lumo FlyWheel Contributors, “FR13 vLLM source of truth and grounding rule.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/FR13_VLLM_SOURCE_OF_TRUTH.md, 2026.
- [33] Lumo FlyWheel Contributors, “FR13 forked FA2 tree server launcher.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/scripts/fr13_launch_forked_fa2_tree_server.sh, 2026.
- [34] Lumo FlyWheel Contributors, “FR10 native speed server launcher.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/scripts/fr10_launch_speed_server.sh, 2026.
- [35] Lumo FlyWheel Contributors, “FR13 measurement harness.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/scripts/fr13_measure.py, 2026.
- [36] Lumo FlyWheel Contributors, “FR13 B=4 contamination verdict.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/FR13_WHY_SLOWER_VERDICT.md, 2026.
- [37] Lumo FlyWheel Contributors, “FR13 WY and replay chase plan.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/FR13_WY_CHASE_PLAN_BIND.md, 2026.
- [38] Lumo FlyWheel Contributors, “Stage D committer microbench.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/research/fr13_workflows/stage_d_committer_microbench.md, 2026.
- [39] Lumo FlyWheel Contributors, “FR13 bake-for-all-trees analysis.” https://github.com/MaCoredroid/Lumo_FlyWheel/blob/37c202bb769a5a3bc0d91b34ed74d2abcb4c3580/research/fr13_workflows/bake_all_tree_flags_verdict.md, 2026.